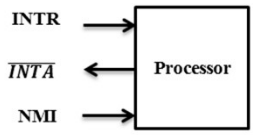
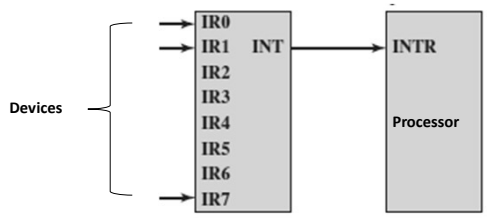
Interrupt Handling in RISC-V

Interrupts in 0x88 machines



Programmable Interrupt Controller



- Intel 82C59A Interrupt Controller
- Internal Priority resolver (priorities are assigned through a combination of hardware and software
- Cascading is possible

3

Receiving Interrupts

- Unlike 8086/88 Intel machines, which receive external interrupts over INTR pin
- RISC-V machines uses an external block called the **PLIC** (Platform-Level Interrupt Controller) or the **CLINT** (Core Local Interruptor).
- These controllers aggregate hundreds of potential interrupt sources.
- When one triggers, the controller sets a specific bit in the CPU's **mip (Machine Interrupt Pending)** register.
- The CPU "receives" the interrupt by constantly monitoring this register bit.

Key Components

- **mip (Machine Interrupt Pending)** : keeps track of pending interrupt requests
- Unlike Intel's external pin-driven acknowledgment, RISC-V manages the process through specific registers:
- **mstatus**: Tracks the global interrupt enable bit (MIE).
- **mie**: A mask register to enable/disable specific interrupt sources (timer, software, external).
- **mtvec**: Holds the base address of the **Interrupt Service Routine (ISR)**.
- **mepc**: Saves the address of the interrupted instruction (not the current one).
- **mcause**: Records the reason for the interrupt (the "exception code").

Interrupt Handling Sequence

Phase A: Hardware Response (Automatic)

- **Completion**: The processor finishes the currently executing instruction.
- **Snapshot**: The address of the next instruction is saved into the **mepc** (Machine Exception Program Counter) register.
- **State Save**: The current privilege mode and interrupt-enable state are saved in the **mstatus** register.
- **Masking**: Global interrupts are automatically disabled (setting **MIE** to 0) to prevent "nested" interrupts from crashing the stack immediately.
- **Vectoring**: The Program Counter (PC) is updated with the address stored in **mtvec**.

Interrupt Handling Sequence

Phase B: Software Response (The ISR)

- **Context Save**: Since RISC-V doesn't automatically push registers to a stack (unlike some CISC architectures), the software must manually save any registers it plans to use (e.g., x1-x31) onto the stack.
- **Service**: The actual code to handle the device (clearing a timer, reading a UART buffer) executes.
- **Context Restore**: The saved registers are popped back from the stack.

Interrupt Handling Sequence

Phase C: Return

- **The mret Instruction**: The ISR ends with the mret (Machine Ret) instruction. This hardware command:
  - Restores the previous interrupt state (re-enabling interrupts by (setting **MIE** to 1)
  - Sets the PC back to the address stored in **mepc**.
  - Execution resumes exactly where it left off.

Comparison

Feature	**Intel (x86)**	**RISC-V**
Reception Signal	Dedicated INTR hardware pin.	Bit set in the mip (Interrupt Pending) CSR.
Address Pickup	Fetched from Data Bus (Vector)	Stored in mtvec register
Context Saving	Hardware pushes some to stack	Software-driven (manual)
Return Command	IRET	mret

Non Maskable Interrupts

- NMIs usually have a **dedicated hardware pin** on the processor core.
- An NMI indicates a potential catastrophe (like memory corruption), it bypasses all the standard "enable/disable" logic.
- It is **immediately** serviced as soon as the current instruction completes, or even mid-instruction if the hardware allows (depending on the implementation).

Handling Non Maskable Interrupts

- **Dedicated Jump Address**: Instead of using mtvec (which is for standard interrupts), most RISC-V implementations jump to a fixed, implementation-defined address in ROM or high-reliability memory.
- **Saving State**: Since an NMI can strike while the CPU is *already* inside a standard ISR, it often uses a separate set of "backup" registers or a dedicated NMI-specific mepc (called nmepc in some extensions) to ensure the original return address isn't lost.
- **The Cause**: The mcause register will be updated with a specific code indicating an NMI (often a platform-specific value).